

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.



LAB REPORT

on

OPERATING SYSTEMS

Submitted by

Devendra P(1BM25CS439)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by **Devendra P (1BM25CS439)**, who is Bonafede student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026. The Lab report has been approved as it satisfies the academic requirements in respect of **OPERATING SYSTEMS - (23CS4PCOPS)** work prescribed for the said degree.

Seema Patil

Associate Professor

Department of CSE

BMSCE, Bengaluru

Dr. Kavitha Sooda

Professor and Head

Department of CSE

BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF c) Priority d) Round Robin	1-3 4-9 10-15 16-19
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	20-23
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	24-28 29-31 32-33
4.	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	34-36 37-39
5.	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	40-44 45-49
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	50-54
7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	55-61

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyze various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program-1

Question:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

- a) FCFS
- b) SJF
- c) Priority
- d) Round Robin (Experiment with different quantum sizes for RR algorithm)

Code:

➤ FCFS:

```
#include<stdio.h>
struct process
{
    int id, at, bt, ct, tat, wt;
};
int main()
{
    int n, current_time = 0;
    int sumTAT = 0, sumWT = 0;
    printf("Devendra P\n");
    printf("1BM25CS439\n\n");

    printf("Enter No. of Process: ");
    scanf("%d", &n);
    struct process p[n];
    for(int i = 0; i < n; i++)
    {
        p[i].id = i + 1;

        printf("Enter AT for Process P%d: ", i + 1);
        scanf("%d", &p[i].at);

        printf("Enter BT for Process P%d: ", i + 1);
        scanf("%d", &p[i].bt);
    }
    for(int i = 0; i < n - 1; i++)
    {
        for(int j = i + 1; j < n; j++)
        {
```

```

        if(p[i].at > p[j].at)
        {
            struct process temp = p[i];
            p[i] = p[j];
            p[j] = temp;
        }
    }
}
for(int i = 0; i < n; i++)
{
    if(current_time < p[i].at)
    {
        current_time = p[i].at;
    }
    p[i].ct = current_time + p[i].bt;
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;

    current_time = p[i].ct;

    sumTAT += p[i].tat;
    sumWT += p[i].wt;
}
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(int i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].id,
        p[i].at,
        p[i].bt,
        p[i].ct,
        p[i].tat,
        p[i].wt);
}
float avgTAT = (float)sumTAT / n;
float avgWT = (float)sumWT / n;

printf("\nAverage TAT : %.2f\n", avgTAT);
printf("Average WT : %.2f\n", avgWT);

return 0;
}

```

Output :

Devendra P
1BM25CS439

Enter No. of Process: 4
Enter AT for Process P1: 0
Enter BT for Process P1: 5
Enter AT for Process P2: 3
Enter BT for Process P2: 6
Enter AT for Process P3: 2
Enter BT for Process P3: 8
Enter AT for Process P4: 5
Enter BT for Process P4: 9

PID	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P3	2	8	13	11	3
P2	3	6	19	16	10
P4	5	9	28	23	14

Average TAT : 13.75
Average WT : 6.75

➤ **SJF:**

```
#include<stdio.h>
struct process
{
    int id, at, bt, ct, tat, wt;
};

int main()
{
    int n, current_time = 0, completed = 0;
    int sumTAT = 0, sumWT = 0;
    printf("Devendra P\n");
    printf("1BM25CS439\n\n");

    printf("Enter No. of Process: ");
    scanf("%d", &n);
    struct process p[n];
    int visited[n];

    for(int i = 0; i < n; i++)
    {
        p[i].id = i + 1;

        printf("Enter AT and BT for P%d: ", i + 1);
        scanf("%d %d", &p[i].at, &p[i].bt);

        visited[i] = 0;
    }
    while(completed < n)
    {
        int idx = -1;
        int min_bt = 9999;

        for(int i = 0; i < n; i++)
        {
            if(p[i].at <= current_time && visited[i] == 0)
            {
                if(p[i].bt < min_bt)
                {
                    min_bt = p[i].bt;
                    idx = i;
                }
            }
        }
    }
}
```

```

    }
}

if(idx != -1)
{
    current_time += p[idx].bt;

    p[idx].ct = current_time;
    p[idx].tat = p[idx].ct - p[idx].at;
    p[idx].wt = p[idx].tat - p[idx].bt;

    sumTAT += p[idx].tat;
    sumWT += p[idx].wt;

    visited[idx] = 1;
    completed++;
}
else
{
    current_time++;
}
}
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(int i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].id,
        p[i].at,
        p[i].bt,
        p[i].ct,
        p[i].tat,
        p[i].wt);
}

float avgTAT = (float)sumTAT / n;
float avgWT = (float)sumWT / n;

printf("\nAverage TAT : %.2f\n", avgTAT);
printf("Average WT : %.2f\n", avgWT);

return 0;
}

```

Output :

```
Devendra P  
1BM25CS439
```

```
Enter No. of Process: 4  
Enter AT and BT for P1: 0 7  
Enter AT and BT for P2: 8 3  
Enter AT and BT for P3: 3 4  
Enter AT and BT for P4: 5 6
```

PID	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	8	3	14	6	3
P3	3	4	11	8	4
P4	5	6	20	15	9

```
Average TAT : 9.00
```

```
Average WT : 4.00
```

```
PS C:\msys64\ucrt64\bin>
```

➤ SRTF:

```
#include<stdio.h>
struct process
{
    int id, at, bt, ct, tat, wt;
};
int main()
{
    int n, current_time = 0, completed = 0;
    int sumTAT = 0, sumWT = 0;

    printf("Devendra P\n");
    printf("1BM25CS439\n\n");
    printf("Enter No. of Process: ");
    scanf("%d", &n);

    struct process p[n];
    int remaining[n];

    for(int i = 0; i < n; i++)
    {
        p[i].id = i + 1;

        printf("Enter AT and BT for P%d: ", i + 1);
        scanf("%d %d", &p[i].at, &p[i].bt);
        remaining[i] = p[i].bt;
    }
    while(completed < n)
    {
        int idx = -1;
        int min = 9999;

        for(int i = 0; i < n; i++)
        {
            if(p[i].at <= current_time && remaining[i] > 0)
            {
                if(remaining[i] < min)
                {
                    min = remaining[i];
                    idx = i;
                }
            }
        }
    }
```

```

    }

    if(idx != -1)
    {
        remaining[idx]--;
        current_time++;

        if(remaining[idx] == 0)
        {
            completed++;

            p[idx].ct = current_time;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;

            sumTAT += p[idx].tat;
            sumWT += p[idx].wt;
        }
    }
    else
    {
        current_time++;
    }
}

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(int i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].id,
        p[i].at,
        p[i].bt,
        p[i].ct,
        p[i].tat,
        p[i].wt);
}

float avgTAT = (float)sumTAT / n;
float avgWT = (float)sumWT / n;

printf("\nAverage TAT : %.2f\n", avgTAT);
printf("Average WT : %.2f\n", avgWT);
return 0;
}

```

Output :

Devendra P
1BM25CS439

Enter No. of Process: 4
Enter AT and BT for P1: 0 7
Enter AT and BT for P2: 8 3
Enter AT and BT for P3: 3 2
Enter AT and BT for P4: 5 6

PID	AT	BT	CT	TAT	WT
P1	0	7	9	9	2
P2	8	3	12	4	1
P3	3	2	5	2	0
P4	5	6	18	13	7

Average TAT : 7.00
Average WT : 2.50

PS C:\msys64\ucrt64\bin> █

➤ **Priority Scheduling – Non Preemptive:**

```
#include<stdio.h>
struct process
{
    int at, id, bt, pr, ct, tat, wt;
};

int main()
{
    int n, current_time = 0, completed = 0;
    float avgTAT = 0, avgWT = 0;

    printf("Devendra P\n");
    printf("1BM25CS439\n\n");

    printf("Enter No. of Process: ");
    scanf("%d", &n);

    struct process p[n];
    int visited[n];

    for(int i = 0; i < n; i++)
    {
        p[i].id = i + 1;

        printf("Enter AT and BT for P%d: ", i + 1);
        scanf("%d %d", &p[i].at, &p[i].bt);

        printf("Enter Priority for P%d: ", i + 1);
        scanf("%d", &p[i].pr);

        visited[i] = 0;
    }
    printf("\nAssuming lowest number as highest priority\n");
    while(completed < n)
    {
```

```

int idx = -1;
int highest = 999;
for(int i = 0; i < n; i++)
{
    if(p[i].at <= current_time && visited[i] == 0)
    {
        if(p[i].pr < highest)
        {
            highest = p[i].pr;
            idx = i;
        }
    }
}

if(idx != -1)
{
    current_time += p[idx].bt;
    p[idx].ct = current_time;
    p[idx].tat = p[idx].ct - p[idx].at;
    p[idx].wt = p[idx].tat - p[idx].bt;
    avgTAT += p[idx].tat;
    avgWT += p[idx].wt;
    visited[idx] = 1;
    completed++;
}
else
{
    current_time++;
}
}
printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");
for(int i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].id,
        p[i].at,
        p[i].bt,

```

```

        p[i].pr,
        p[i].ct,
        p[i].tat,
        p[i].wt);
    }

    avgTAT = avgTAT / n;
    avgWT = avgWT / n;

    printf("\nAverage TAT : %.2f\n", avgTAT);
    printf("Average WT : %.2f\n", avgWT);

    return 0;
}

```

Output :

```

Devendra P
1BM25CS439

Enter No. of Process: 5
Enter AT and BT for P1: 0 3
Enter Priority for P1: 5
Enter AT and BT for P2: 2 2
Enter Priority for P2: 3
Enter AT and BT for P3: 3 5
Enter Priority for P3: 2
Enter AT and BT for P4: 4 4
Enter Priority for P4: 4
Enter AT and BT for P5: 6 1
Enter Priority for P5: 1

Assuming lowest number as highest priority

PID    AT    BT    PR    CT    TAT    WT
1       0     3     5     3     3     0
2       2     2     3     11    9     7
3       3     5     2     8     5     0
4       4     4     4     15    11    7
5       6     1     1     9     3     2

Average TAT : 6.20
Average WT  : 3.20

```

➤ Priority Scheduling – Preemptive:

```
#include<stdio.h>
struct process
{
    int id, at, bt, pr, ct, tat, wt;
};

int main()
{
    int n, current_time = 0, completed = 0;
    float avgTAT = 0, avgWT = 0;

    printf("Devendra P\n");
    printf("1BM25CS439\n\n");

    printf("Enter No. of Process: ");
    scanf("%d", &n);

    struct process p[n];
    int remaining[n];

    for(int i = 0; i < n; i++)
    {
        p[i].id = i + 1;

        printf("Enter AT and BT for P%d: ", i + 1);
        scanf("%d %d", &p[i].at, &p[i].bt);

        printf("Enter Priority for P%d: ", i + 1);
        scanf("%d", &p[i].pr);

        remaining[i] = p[i].bt;
    }
    printf("\nAssuming lower number = higher priority\n");

    while(completed < n)
    {
        int idx = -1;
        int highest = 9999;

        for(int i = 0; i < n; i++)
        {
```

```

if(p[i].at <= current_time && remaining[i] > 0)
{
    // Select higher priority process
    if(p[i].pr < highest)
    {
        highest = p[i].pr;
        idx = i;
    }
    // Tie-breaking using arrival time
    else if(p[i].pr == highest)
    {
        if(p[i].at < p[idx].at)
        {
            idx = i;
        }
    }
}

// If process found
if(idx != -1)
{
    remaining[idx]--;
    current_time++;

    // Process completed
    if(remaining[idx] == 0)
    {
        completed++;
        p[idx].ct = current_time;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;
        avgTAT += p[idx].tat;
        avgWT += p[idx].wt;
    }
}
else
{
    // CPU Idle
    current_time++;
}
}
avgTAT /= n;
avgWT /= n;

```

```

printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(int i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].id,
        p[i].at,
        p[i].bt,
        p[i].pr,
        p[i].ct,
        p[i].tat,
        p[i].wt);
}
printf("\nAverage TAT : %.2f\n", avgTAT);
printf("Average WT : %.2f\n", avgWT);

return 0;
}

```

Output :

```

Devendra P
1BM25CS439

Enter No. of Process: 5
Enter AT and BT for P1: 0 3
Enter Priority for P1: 5
Enter AT and BT for P2: 2 2
Enter Priority for P2: 3
Enter AT and BT for P3: 3 5
Enter Priority for P3: 2
Enter AT and BT for P4: 4 4
Enter Priority for P4: 4
Enter AT and BT for P5: 6 1
Enter Priority for P5: 1

Assuming lower number = higher priority

PID    AT    BT    PR    CT    TAT    WT
P1      0     3     5    15    15    12
P2      2     2     3    10     8     6
P3      3     5     2     9     6     1
P4      4     4     4    14    10     6
P5      6     1     1     7     1     0

Average TAT : 8.00
Average WT  : 5.00

```

➤ **Round Robin :**

```
#include<stdio.h>
struct process
{
    int id, at, bt, remaining, completion, wt, tat;
};

int main()
{
    int n, tq;

    printf("Devendra P\n");
    printf("1BM25CS439\n\n");

    printf("Enter No. of Process: ");
    scanf("%d", &n);

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    struct process p[n];

    for(int i = 0; i < n; i++)
    {
        p[i].id = i + 1;

        printf("\nProcess P%d\n", p[i].id);

        printf("Arrival Time: ");
        scanf("%d", &p[i].at);

        printf("Burst Time: ");
        scanf("%d", &p[i].bt);

        p[i].remaining = p[i].bt;
        p[i].completion = 0;
```

```

    p[i].wt = 0;
    p[i].tat = 0;
}

int current = 0, completed = 0;

while(completed < n)
{
    int executed = 0;

    for(int i = 0; i < n; i++)
    {
        if(p[i].at <= current && p[i].remaining > 0)
        {
            executed = 1;

            if(p[i].remaining > tq)
            {
                current += tq;
                p[i].remaining -= tq;
            }
            else
            {
                current += p[i].remaining;

                p[i].remaining = 0;

                p[i].completion = current;

                p[i].tat = p[i].completion - p[i].at;

                p[i].wt = p[i].tat - p[i].bt;

                completed++;
            }
        }
    }
}

```

```

        if(executed == 0)
        {
            current++;
        }
    }

    float avgTAT = 0, avgWT = 0;

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

    for(int i = 0; i < n; i++)
    {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].id,
            p[i].at,
            p[i].bt,
            p[i].completion,
            p[i].tat,
            p[i].wt);

        avgTAT += p[i].tat;
        avgWT += p[i].wt;
    }

    avgTAT = avgTAT / n;
    avgWT = avgWT / n;

    printf("\nAverage TAT : %.2f\n", avgTAT);
    printf("Average WT : %.2f\n", avgWT);

    return 0;
}

```

Output :

```
Devendra P
1BM25CS439

Enter No. of Process: 5
Enter Time Quantum: 2

Process P1
Arrival Time: 0
Burst Time: 5

Process P2
Arrival Time: 1
Burst Time: 3

Process P3
Arrival Time: 2
Burst Time: 1

Process P4
Arrival Time: 3
Burst Time: 2

Process P5
Arrival Time: 4
Burst Time: 3

PID    AT    BT    CT    TAT    WT
1       0     5     14    14     9
2       1     3     12    11     8
3       2     1     5     3      2
4       3     2     7     4      2
5       4     3     13    9      6

Average TAT : 8.20
Average WT  : 5.40
```

Program-2

Question:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Code:

```
#include<stdio.h>
#define MAX 10
struct process
{
    int pid, at, bt, rt, ct, tat, wt, queue;
};
int main()
{
    struct process p[MAX];

    int n, i, time = 0, completed = 0;
    int tq = 2;
    printf("Devendra P\n");
    printf("1BM25CS439\n\n");
    printf("Enter number of Process: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        printf("\nEnter details of P%d\n", i + 1);
        p[i].pid = i + 1;
        printf("Arrival Time: ");
        scanf("%d", &p[i].at);

        printf("Burst Time: ");
        scanf("%d", &p[i].bt);

        printf("Queue (1-System, 2-User): ");
        scanf("%d", &p[i].queue);

        p[i].rt = p[i].bt;
    }
    printf("\nEnter Time Quantum (for system process): ");
    scanf("%d", &tq);
    printf("\nGANTT CHART:\n");
```

```

while(completed < n)
{
    int executed = 0;

    for(i = 0; i < n; i++)
    {
        if(p[i].queue == 1 && p[i].rt > 0 && p[i].at <= time)
        {
            executed = 1;

            if(p[i].rt > tq)
            {
                printf("P%d ", p[i].pid);

                time += tq;

                p[i].rt -= tq;
            }
            else
            {
                printf("P%d ", p[i].pid);

                time += p[i].rt;

                p[i].rt = 0;

                p[i].ct = time;

                completed++;
            }
        }
    }
}

// Queue 2 -> FCFS
if(!executed)
{
    for(i = 0; i < n; i++)
    {
        if(p[i].queue == 2 && p[i].rt > 0 && p[i].at <= time)
        {
            executed = 1;

            printf("P%d ", p[i].pid);

```

```

        time += p[i].rt;

        p[i].rt = 0;

        p[i].ct = time;

        completed++;

        break;
    }
}

if(!executed)
{
    time++;
}
}

float total_tat = 0, total_wt = 0;
printf("\n\nPID\tAT\tBT\tType\tCT\tTAT\tWT\n");
for(i = 0; i < n; i++)
{
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
    total_tat += p[i].tat;
    total_wt += p[i].wt;
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,
        p[i].at,
        p[i].bt,
        p[i].queue,
        p[i].ct,
        p[i].tat,
        p[i].wt);
}
printf("\nAverage Turnaround Time = %.2f",
    total_tat / n);
printf("\nAverage Waiting Time = %.2f\n",
    total_wt / n);
return 0;
}

```

Output :

```
Devendra P
1BM25CS439

Enter number of Process: 4

Enter details of P1
Arrival Time: 0
Burst Time: 3
Queue (1-System, 2-User): 1

Enter details of P2
Arrival Time: 0
Burst Time: 4
Queue (1-System, 2-User): 2

Enter details of P3
Arrival Time: 2
Burst Time: 3
Queue (1-System, 2-User): 1

Enter details of P4
Arrival Time: 2
Burst Time: 5
Queue (1-System, 2-User): 2

Enter Time Quantum (for system process): 2

GANTT CHART:
P1 P3 P1 P3 P2 P4

PID    AT    BT    Type    CT    TAT    WT
1       0     3     1       5     5      2
2       0     4     2      10    10     6
3       2     3     1       6     4      1
4       2     5     2      15    13     8

Average Turnaround Time = 8.00
Average Waiting Time = 4.25
```

Program-3

Question:

Write a C program to simulate Real-Time CPU Scheduling algorithms:

- a) Rate- Monotonic
- b) Earliest-deadline First
- c) Proportional scheduling

Code:

➤ Rate- Monotonic:

```
#include<stdio.h>
#include<math.h>
struct process
{
    int pid, bt, period, remaining_burst;
};
int gcd(int a, int b)
{
    if(b == 0)
        return a;

    return gcd(b, a % b);
}

int lcm(int a, int b)
{
    return (a * b) / gcd(a, b);
}
int main()
{
    int n;

    printf("Devendra P\n");
    printf("1BM25CS439\n\n");

    printf("Enter the no of processes: ");
    scanf("%d", &n);
```

```

struct process p[n];

printf("Enter CPU Burst Time:\n");

for(int i = 0; i < n; i++)
{
    scanf("%d", &p[i].bt);
    p[i].pid = i + 1;
}

printf("Enter Time Period:\n");

int hyperperiod = 1;
double utilization = 0;

for(int i = 0; i < n; i++)
{
    scanf("%d", &p[i].period);

    p[i].remaining_burst = 0;

    hyperperiod = lcm(hyperperiod, p[i].period);

    utilization += (double)p[i].bt / p[i].period;
}

// Sort according to period (smaller period = higher priority)
for(int i = 0; i < n; i++)
{
    for(int j = i + 1; j < n; j++)
    {
        if(p[i].period > p[j].period)
        {
            struct process temp = p[i];
            p[i] = p[j];
            p[j] = temp;
        }
    }
}

```

```

    }
}

printf("\nHyperperiod (LCM) = %d\n", hyperperiod);

printf("\nRate Monotonic Scheduling\n");

printf("PID\tBT\tPeriod\n");

for(int i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\n",
        p[i].pid,
        p[i].bt,
        p[i].period);
}

double bound = n * (pow(2.0, 1.0 / n) - 1.0);
printf("\nUtilization = %.3lf\n", utilization);
printf("Bound      = %.3lf\n", bound);
if(utilization <= 1)
{
    printf("\nSchedulable\n");
    printf("\nScheduling for %d ms:\n\n", hyperperiod);
    int current_process = -1;
    for(int time = 0; time < hyperperiod; time++)
    {
        // Process release
        for(int i = 0; i < n; i++)
        {
            if(time % p[i].period == 0)
            {
                // Deadline miss check
                if(p[i].remaining_burst > 0)
                {
                    printf("Deadline Miss for Process P%d at time %d\n",

```

```

        p[i].pid, time);
    }

    p[i].remaining_burst += p[i].bt;
}
}
int next_process = -1;

// Highest priority ready process
for(int i = 0; i < n; i++)
{
    if(p[i].remaining_burst > 0)
    {
        next_process = i;
        break;
    }
}
// Context switch display
if(next_process != current_process)
{
    if(next_process == -1)
    {
        printf("%d ms onwards : CPU Idle\n", time);
    }
    else
    {
        printf("%d ms onwards : Process P%d running\n",
            time,
            p[next_process].pid);
    }

    current_process = next_process;
}

// Execute process
if(current_process != -1)
{

```

```

        p[current_process].remaining_burst--;
    }
}
else
{
    printf("\nNot Schedulable\n");
}
return 0;
}

```

Output :

```

Devendra P
1BM25CS439

Enter the no of processes: 3
Enter CPU Burst Time:
1 2 3
Enter Time Period:
4 6 12

Hyperperiod (LCM) = 12

Rate Monotonic Scheduling
PID    BT    Period
P1      1      4
P2      2      6
P3      3     12

Utilization = 0.833
Bound       = 0.780

Schedulable

Scheduling for 12 ms:

0 ms onwards : Process P1 running
1 ms onwards : Process P2 running
3 ms onwards : Process P3 running
4 ms onwards : Process P1 running
5 ms onwards : Process P3 running
6 ms onwards : Process P2 running
8 ms onwards : Process P1 running
9 ms onwards : Process P3 running
10 ms onwards : CPU Idle

```

➤ **Earliest-deadline First:**

```
#include<stdio.h>
#include<math.h>
struct process
{
    int pid, burst, deadline, period;
    int remaining_burst, absolute_deadline;
};

int gcd(int a, int b)
{
    if(b == 0)
        return a;

    return gcd(b, a % b);
}

int lcm(int a, int b)
{
    return (a * b) / gcd(a, b);
}

int main()
{
    int n, hyperperiod = 1;

    printf("Devendra P\n");
    printf("1BM25CS439\n\n");

    printf("Enter the no of process: ");
    scanf("%d", &n);
    struct process p[n];
    for(int i = 0; i < n; i++)
    {
        p[i].pid = i + 1;
        printf("\nProcess %d - Enter Burst, Deadline, Period: ",
            i + 1);
        scanf("%d %d %d",
            &p[i].burst,
            &p[i].deadline,
            &p[i].period);
    }
}
```

```

    p[i].remaining_burst = 0;

    hyperperiod = lcm(hyperperiod, p[i].period);
}
printf("\nPID\tBurst\tDeadline\tPeriod\n");

for(int i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\t%d\n",
        p[i].pid,
        p[i].burst,
        p[i].deadline,
        p[i].period);
}
printf("\nScheduling occurs for %d ms\n\n",
    hyperperiod);

for(int time = 0; time < hyperperiod; time++)
{
    for(int i = 0; i < n; i++)
    {
        if(time % p[i].period == 0)
        {
            p[i].remaining_burst = p[i].burst;

            p[i].absolute_deadline =
                time + p[i].deadline;
        }
    }
    int min_deadline = 99999;
    int selected = -1;

    for(int i = 0; i < n; i++)
    {
        if(p[i].remaining_burst > 0)
        {
            if(p[i].absolute_deadline < min_deadline)
            {
                min_deadline =
                    p[i].absolute_deadline;

                selected = i;
            }
        }
    }
}

```

```

    }
    if(selected == -1)
    {
        printf("%d ms : CPU is idle\n",
            time);
    }
    else
    {
        printf("%d ms : Task %d is running\n",
            time,
            p[selected].pid);
        p[selected].remaining_burst--;
    }
}
return 0;
}

```

Output :

```

Devendra P
18M25CS439

Enter the no of process: 3

Process 1 - Enter Burst, Deadline, Period: 3 7 20
Process 2 - Enter Burst, Deadline, Period: 2 4 5
Process 3 - Enter Burst, Deadline, Period: 2 8 10

PID    Burst    Deadline    Period
1       3         7           20
2       2         4           5
3       2         8           10

Scheduling occurs for 20 ms

0 ms : Task 2 is running
1 ms : Task 2 is running
2 ms : Task 1 is running
3 ms : Task 1 is running
4 ms : Task 1 is running
5 ms : Task 3 is running
6 ms : Task 3 is running
7 ms : Task 2 is running
8 ms : Task 2 is running
9 ms : CPU is idle
10 ms : Task 2 is running
11 ms : Task 2 is running
12 ms : Task 3 is running
13 ms : Task 3 is running
14 ms : CPU is idle
15 ms : Task 2 is running
16 ms : Task 2 is running
17 ms : CPU is idle
18 ms : CPU is idle
19 ms : CPU is idle

```

➤ **Proportional scheduling:**

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
typedef struct
{
    char name[5];
    int tickets;
} Process;
int main()
{
    int n, total_tickets = 0;
    float total_T = 0.0;
    printf("Devendra P\n");
    printf("1BM25CS439\n\n");
    printf("Enter the number of Processes: ");
    scanf("%d", &n);
    Process p[n];
    srand(time(NULL));
    for (int i = 0; i < n; i++)
    {
        printf("\nProcess %d:\n", i + 1);
        sprintf(p[i].name, "P%d", i + 1);
        printf("Tickets: ");
        scanf("%d", &p[i].tickets);
        total_tickets += p[i].tickets;
        total_T += p[i].tickets;
    }
    printf("\n--- Proportional Share Scheduling ---\n");
    printf("Enter the Time Period for scheduling: ");
    int m;
    scanf("%d", &m);
    for (int i = 0; i < m; i++)
    {
        int winning_ticket = rand() % total_tickets + 1;
        int accumulated_tickets = 0;
```

```

int winner_index;
for (int j = 0; j < n; j++)
{
    accumulated_tickets += p[j].tickets;
    if (winning_ticket <= accumulated_tickets)
    {
        winner_index = j;
        break;
    }
}

printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
       p[winner_index].name);
}
for (int i = 0; i < n; i++)
{
    printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name,
           ((p[i].tickets / total_T) * 100));
}
return 0;
}

```

Output :

```

Devendra P
1BM25CS439

Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 3, Winner: P1
Tickets picked: 14, Winner: P2
Tickets picked: 59, Winner: P3
Tickets picked: 45, Winner: P3
Tickets picked: 19, Winner: P2

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.
PS C:\msys64\ucrt64\bin>

```

Program-4

Question:

Write a C program to simulate:

- a) Producer-Consumer problem using semaphores.
- b) Dining-Philosopher's problem

Code:

➤ Producer-Consumer problem :

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>
#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty;      // counts empty slots
sem_t full;        // counts filled slots
pthread_mutex_t mutex; // for mutual exclusion

void *producer(void *arg)
{
    int item, i = 0;
    while (1)
    {
        item = i++;          // produce an item (here just incrementing number)
        sem_wait(&empty);    // wait for empty slot
        pthread_mutex_lock(&mutex); // enter critical section
        buffer[in] = item;
        printf("Produced: %d at buffer[%d]\n", item, in);
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex); // leave critical section
        sem_post(&full);           // signal that new item is available
        sleep(1);                  // simulate time to produce
    }
}
```

```

void *consumer(void *arg)
{
    int item;
    while (1)
    {
        sem_wait(&full);          // wait for filled slot
        pthread_mutex_lock(&mutex); // enter critical section
        item = buffer[out];
        printf("Consumed: %d from buffer[%d]\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex); // leave critical section
        sem_post(&empty);           // signal that a slot is free
        sleep(2);                   // simulate time to consume
    }
}

int main()
{
    pthread_t prod, cons;
    printf("Devendra P \n1BM25CS439\n");
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
    return 0;
}

```

Output :

```
Devendra P
IBM25CS439
Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Produced: 2 at buffer[2]
Consumed: 1 from buffer[1]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Produced: 7 at buffer[2]
Consumed: 4 from buffer[4]
Produced: 8 at buffer[3]
Produced: 9 at buffer[4]
Consumed: 5 from buffer[0]
Produced: 10 at buffer[0]
Consumed: 6 from buffer[1]
Produced: 11 at buffer[1]
Consumed: 7 from buffer[2]
Produced: 12 at buffer[2]
Consumed: 8 from buffer[3]
Produced: 13 at buffer[3]
Consumed: 9 from buffer[4]
Produced: 14 at buffer[4]
Consumed: 10 from buffer[0]
Produced: 15 at buffer[0]
```

➤ Dining-Philosopher's problem :

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define N 5          // Number of philosophers
pthread_mutex_t forks[N]; // One mutex per fork
pthread_t philosophers[N];

void *philosopher(void *num)
{
    int id = *(int *)num;
    int left = id;          // left fork index
    int right = (id + 1) % N; // right fork index
    while (1)
    {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1); // Thinking
        // To avoid deadlock, philosophers with even id pick left then right,
        // odd id pick right then left.
        if (id % 2 == 0)
        {
            // Pick up left fork
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);
            // Pick up right fork

            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);
        }
        else
        {
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right); // Pick up left
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);
        }
        printf("Philosopher %d is eating.\n", id);
        sleep(2); // Put down forks
        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);
        printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
    }
}
```

```

    return NULL;
}
int main()
{
    int i;
    int ids[N]; // Initialize forks (mutexes)
    printf("Devendra P\n1BM25CS439\n");
    for (i = 0; i < N; i++)
    {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }
    for (i = 0; i < N; i++)
    {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    } // Join threads (will never happen here, but good practice)
    for (i = 0; i < N; i++)
    {
        pthread_join(philosophers[i], NULL);
    }
    // Destroy mutexes (unreachable here)
    for (i = 0; i < N; i++)
    {
        pthread_mutex_destroy(&forks[i]);
    }
    return 0;
}

```

Output :

```
Devendra P
18M25CS439
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 2 picked up left fork 2.
Philosopher 4 picked up left fork 4.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 4 put down forks 4 and 0.
Philosopher 4 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 3 put down forks 3 and 4.
Philosopher 3 is thinking.
```

Program-5

Question:

Write a C program to simulate:

- a) Bankers' algorithm for the purpose of deadlock avoidance.
- b) Deadlock Detection

Code:

➤ Bankers' algorithm:

```
#include <stdio.h>
#include <stdbool.h>

#define MAX_PROCESSES 10
#define MAX_RESOURCES 10

int main()
{
    int n, m;

    printf("Devendra P\n");
    printf("1BM25CS439\n\n");

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resource types: ");
    scanf("%d", &m);

    int allocation[MAX_PROCESSES][MAX_RESOURCES];
    int max[MAX_PROCESSES][MAX_RESOURCES];
    int need[MAX_PROCESSES][MAX_RESOURCES];
    int available[MAX_RESOURCES];

    printf("\nEnter Allocation Matrix:\n");
    for(int i = 0; i < n; i++)
    {
```

```

    for(int j = 0; j < m; j++)
    {
        scanf("%d", &allocation[i][j]);
    }
}

printf("\nEnter Max Matrix:\n");
for(int i = 0; i < n; i++)
{
    for(int j = 0; j < m; j++)
    {
        scanf("%d", &max[i][j]);
    }
}

printf("\nEnter Available Resources:\n");
for(int j = 0; j < m; j++)
{
    scanf("%d", &available[j]);
}

// Calculate Need Matrix
for(int i = 0; i < n; i++)
{
    for(int j = 0; j < m; j++)
    {
        need[i][j] = max[i][j] - allocation[i][j];
    }
}

printf("\nNeed Matrix:\n");
for(int i = 0; i < n; i++)
{
    for(int j = 0; j < m; j++)
    {
        printf("%d ", need[i][j]);
    }
}

```

```

    printf("\n");
}

bool finish[MAX_PROCESSES] = {false};
int safeSequence[MAX_PROCESSES];
int work[MAX_RESOURCES];

for(int j = 0; j < m; j++)
{
    work[j] = available[j];
}

int count = 0;

while(count < n)
{
    bool found = false;

    for(int i = 0; i < n; i++)
    {
        if(!finish[i])
        {
            bool canExecute = true;

            for(int j = 0; j < m; j++)
            {
                if(need[i][j] > work[j])
                {
                    canExecute = false;
                    break;
                }
            }

            if(canExecute)
            {
                for(int j = 0; j < m; j++)
                {

```

```

        work[j] += allocation[i][j];
    }

    safeSequence[count] = i;
    count++;

    finish[i] = true;
    found = true;
}
}
}

if(!found)
{
    break;
}
}
if(count == n)
{
    printf("\nSystem is in SAFE State.\n");
    printf("Safe Sequence: ");
    for(int i = 0; i < n; i++)
    {
        printf("P%d", safeSequence[i]);
        if(i != n - 1)
        {
            printf(" -> ");
        }
    }
    printf("\n");
}
else
{
    printf("\nSystem is NOT in Safe State.\n");
}
return 0;
}

```

Output :

```
Devendra P
1BM25CS439

Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Max Matrix:

7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

Need Matrix:
7 4 3
1 2 2
6 0 0
0 1 1
4 3 1

System is in SAFE State.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
```

➤ Deadlock Detection:

```
#include <stdio.h>
#include <stdbool.h>

#define MAX_PROCESSES 10
#define MAX_RESOURCES 10

int main()
{
    int n, m;

    printf("Devendra P\n");
    printf("1BM25CS439\n\n");

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resource types: ");
    scanf("%d", &m);

    int allocation[MAX_PROCESSES][MAX_RESOURCES];
    int request[MAX_PROCESSES][MAX_RESOURCES];
    int available[MAX_RESOURCES];

    printf("\nEnter Allocation Matrix:\n");
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < m; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("\nEnter Request Matrix:\n");
    for(int i = 0; i < n; i++)
    {
```

```

    for(int j = 0; j < m; j++)
    {
        scanf("%d", &request[i][j]);
    }
}

printf("\nEnter Available Resources:\n");
for(int j = 0; j < m; j++)
{
    scanf("%d", &available[j]);
}

bool finish[MAX_PROCESSES];
int work[MAX_RESOURCES];
int safeSequence[MAX_PROCESSES];

for(int j = 0; j < m; j++)
{
    work[j] = available[j];
}

for(int i = 0; i < n; i++)
{
    bool zeroAllocation = true;

    for(int j = 0; j < m; j++)
    {
        if(allocation[i][j] != 0)
        {
            zeroAllocation = false;
            break;
        }
    }

    finish[i] = zeroAllocation;
}

```

```

int count = 0;

while(1)
{
    bool found = false;

    for(int i = 0; i < n; i++)
    {
        if(!finish[i])
        {
            bool canExecute = true;

            for(int j = 0; j < m; j++)
            {
                if(request[i][j] > work[j])
                {
                    canExecute = false;
                    break;
                }
            }

            if(canExecute)
            {
                for(int j = 0; j < m; j++)
                {
                    work[j] += allocation[i][j];
                }

                finish[i] = true;

                safeSequence[count] = i;
                count++;

                found = true;
            }
        }
    }
}

```

```

        if(!found)
        {
            break;
        }
    }
    bool deadlock = false;

    for(int i = 0; i < n; i++)
    {
        if(!finish[i])
        {
            deadlock = true;
            break;
        }
    }
    if(!deadlock)
    {
        printf("\nNo Deadlock Detected.\n");

        printf("Safe Sequence: ");

        for(int i = 0; i < count; i++)
        {
            printf("P%d", safeSequence[i]);

            if(i != count - 1)
            {
                printf(" -> ");
            }
        }

        printf("\n");
    }
    else
    {
        printf("\nDeadlock Detected.\n");
    }

```

```

printf("Processes involved in deadlock: ");

for(int i = 0; i < n; i++)
{
    if(!finish[i])
    {
        printf("P%d ", i);
    }
}

printf("\n");
}
return 0;
}

```

Output :

```

Devendra P
1BM25CS439

Enter number of processes: 4
Enter number of resource types: 3

Enter Allocation Matrix:
1 0 2
2 1 1
1 0 3
1 2 2

Enter Request Matrix:
0 0 1
1 0 2
0 0 0
3 3 3

Enter Available Resources:
0 0 0

Deadlock Detected.
Processes involved in deadlock: P3

```

Program-6

Question:

Write a C program to simulate the following contiguous memory allocation techniques.

- a) Worst-fit
- b) Best-fit
- c) First-fit

Code:

```
#include <stdio.h>
void firstFit(int blocks[], int b, int processes[], int p)
{
    int allocation[10];
    int used[10] = {0};
    int i, j;

    for(i = 0; i < p; i++)
        allocation[i] = -1;

    for(i = 0; i < p; i++)
    {
        for(j = 0; j < b; j++)
        {
            // Check if block fits and is unused
            if(blocks[j] >= processes[i] && used[j] == 0)
            {
                allocation[i] = j;
                used[j] = 1;
                break;
            }
        }
    }
}

printf("\n--- First Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for(i = 0; i < p; i++)
{
    printf("%d\t%d\t", i + 1, processes[i]);

    if(allocation[i] != -1)
```

```

        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
    }
}
void bestFit(int blocks[], int b, int processes[], int p)
{
    int allocation[10];
    int used[10] = {0};
    int i, j;

    for(i = 0; i < p; i++)
        allocation[i] = -1;

    for(i = 0; i < p; i++)
    {
        int best = -1;

        for(j = 0; j < b; j++)
        {
            if(blocks[j] >= processes[i] && used[j] == 0)
            {
                if(best == -1 || blocks[j] < blocks[best])
                {
                    best = j;
                }
            }
        }

        if(best != -1)
        {
            allocation[i] = best;
            used[best] = 1;
        }
    }

    printf("\n--- Best Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");

    for(i = 0; i < p; i++)
    {
        printf("%d\t\t%d\t\t", i + 1, processes[i]);

        if(allocation[i] != -1)

```

```

        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
    }
}
void worstFit(int blocks[], int b, int processes[], int p)
{
    int allocation[10];
    int used[10] = {0};
    int i, j;

    for(i = 0; i < p; i++)
        allocation[i] = -1;

    for(i = 0; i < p; i++)
    {
        int worst = -1;

        for(j = 0; j < b; j++)
        {
            if(blocks[j] >= processes[i] && used[j] == 0)
            {
                if(worst == -1 || blocks[j] > blocks[worst])
                {
                    worst = j;
                }
            }
        }

        if(worst != -1)
        {
            allocation[i] = worst;
            used[worst] = 1;
        }
    }

    printf("\n--- Worst Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");

    for(i = 0; i < p; i++)
    {
        printf("%d\t\t%d\t\t", i + 1, processes[i]);

        if(allocation[i] != -1)

```

```

        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
    }
}
int main()
{
    int blocks[10], processes[10];
    int b, p, i;

    printf("Devendra P\n");
    printf("1BM25CS439\n\n");

    // Input number of blocks
    printf("Enter number of memory blocks: ");
    scanf("%d", &b);

    // Input block sizes
    printf("Enter sizes of %d memory blocks:\n", b);

    for(i = 0; i < b; i++)
    {
        scanf("%d", &blocks[i]);
    }

    printf("Enter number of processes: ");
    scanf("%d", &p);
    printf("Enter sizes of %d processes:\n", p);

    for(i = 0; i < p; i++)
    {
        scanf("%d", &processes[i]);
    }
    firstFit(blocks, b, processes, p);
    bestFit(blocks, b, processes, p);
    worstFit(blocks, b, processes, p);

    return 0;
}

```

Output :

```
Devendra P
1BM25CS439

Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

--- First Fit ---
Process No.    Process Size    Block No.
1              212          2
2              417          5
3              112          3
4              426        Not Allocated

--- Best Fit ---
Process No.    Process Size    Block No.
1              212          4
2              417          2
3              112          3
4              426          5

--- Worst Fit ---
Process No.    Process Size    Block No.
1              212          5
2              417          2
3              112          4
4              426        Not Allocated
```

Program-7

Question:

Write a C program to simulate page replacement algorithms.

- a) FIFO
- b) LRU
- c) Optimal

Code:

```
#include <stdio.h>
void printFrames(int frame[], int frames) {
    for(int i = 0; i < frames; i++) {
        if(frame[i] == -1)
            printf("- ");
        else
            printf("%d ", frame[i]);
    }
}
```

// FIFO Algorithm

```
void fifo(int pages[], int n, int frames) {
```

```
    int frame[frames];
    int faults = 0, hits = 0;
```

```
    for(int i = 0; i < frames; i++)
        frame[i] = -1;
```

```
    int index = 0;
```

```
    printf("\nFIFO:\n");
```

```
    for(int i = 0; i < n; i++) {
```

```
        int found = 0;
```

```
        // Check page hit
```

```
        for(int j = 0; j < frames; j++) {
            if(frame[j] == pages[i]) {
                found = 1;
                hits++;
            }
        }
    }
```

```

        break;
    }
}

// Page Fault
if(!found) {
    frame[index] = pages[i];
    index = (index + 1) % frames;
    faults++;
}

printf("Page %d -> ", pages[i]);
printFrames(frame, frames);

if(found)
    printf(" HIT");
else
    printf(" FAULT");

printf("\n");
}

printf("\nTotal Page Faults = %d\n", faults);
printf("Total Page Hits  = %d\n", hits);
}

// LRU Algorithm
void lru(int pages[], int n, int frames) {

    int frame[frames], time[frames];
    int faults = 0, hits = 0, counter = 0;

    for(int i = 0; i < frames; i++) {
        frame[i] = -1;
        time[i] = -1;
    }

    printf("\nLRU:\n");

    for(int i = 0; i < n; i++) {

        int found = 0;

        // Check page hit

```

```

for(int j = 0; j < frames; j++) {
    if(frame[j] == pages[i]) {
        counter++;
        time[j] = counter;
        found = 1;
        hits++;
        break;
    }
}

// Page Fault
if(!found) {

    int pos = 0;

    // Find empty frame first
    for(int j = 0; j < frames; j++) {

        if(frame[j] == -1) {
            pos = j;
            break;
        }

        if(time[j] < time[pos])
            pos = j;
    }

    frame[pos] = pages[i];

    counter++;
    time[pos] = counter;

    faults++;
}

printf("Page %d -> ", pages[i]);
printFrames(frame, frames);

if(found)
    printf(" HIT");
else
    printf(" FAULT");

printf("\n");

```

```

    }

    printf("\nTotal Page Faults = %d\n", faults);
    printf("Total Page Hits  = %d\n", hits);
}

```

// Optimal Algorithm

```
void optimal(int pages[], int n, int frames) {
```

```
    int frame[frames];
    int faults = 0, hits = 0;
```

```
    for(int i = 0; i < frames; i++)
        frame[i] = -1;
```

```
    printf("\nOptimal:\n");
```

```
    for(int i = 0; i < n; i++) {
```

```
        int found = 0;
```

```
        // Check page hit
```

```
        for(int j = 0; j < frames; j++) {
            if(frame[j] == pages[i]) {
                found = 1;
                hits++;
                break;
            }
        }
    }
```

```
        // Page Fault
```

```
        if(!found) {
```

```
            int pos = -1;
```

```
            // Check empty frame
```

```
            for(int j = 0; j < frames; j++) {
                if(frame[j] == -1) {
                    pos = j;
                    break;
                }
            }
        }
```

```
        // Apply Optimal Replacement
```

```

if(pos == -1) {

    int farthest = -1;

    for(int j = 0; j < frames; j++) {

        int k;

        // Find next use
        for(k = i + 1; k < n; k++) {
            if(frame[j] == pages[k])
                break;
        }

        // Never used again
        if(k == n) {
            pos = j;
            break;
        }

        // Select farthest page
        if(k > farthest) {
            farthest = k;
            pos = j;
        }
    }

    frame[pos] = pages[i];
    faults++;
}

printf("Page %d -> ", pages[i]);
printFrames(frame, frames);

if(found)
    printf(" HIT");
else
    printf(" FAULT");

printf("\n");
}

printf("\nTotal Page Faults = %d\n", faults);

```

```

    printf("Total Page Hits  = %d\n", hits);
}

int main() {

    int n, frames;

    printf("Devendra P\n");
    printf("1BM25CS439\n\n");

    printf("Enter number of pages: ");
    scanf("%d", &n);

    int pages[n];

    printf("Enter page reference string:\n");

    for(int i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &frames);

    fifo(pages, n, frames);
    lru(pages, n, frames);
    optimal(pages, n, frames);

    return 0;
}

```

Output :

Devendra P
18M25CS439

Enter number of pages: 21
Enter page reference string:
5 0 1 0 2 3 0 2 4 3 3 2 0 2 1 2 7 0 1 1 0
Enter number of frames: 3

FIFO:

Page 5 -> 5 - - FAULT
Page 0 -> 5 0 - FAULT
Page 1 -> 5 0 1 FAULT
Page 0 -> 5 0 1 HIT
Page 2 -> 2 0 1 FAULT
Page 3 -> 2 3 1 FAULT
Page 0 -> 2 3 0 FAULT
Page 2 -> 2 3 0 HIT
Page 4 -> 4 3 0 FAULT
Page 3 -> 4 3 0 HIT
Page 3 -> 4 3 0 HIT
Page 2 -> 4 2 0 FAULT
Page 0 -> 4 2 0 HIT
Page 2 -> 4 2 0 HIT
Page 1 -> 4 2 1 FAULT
Page 2 -> 4 2 1 HIT
Page 7 -> 7 2 1 FAULT
Page 0 -> 7 0 1 FAULT
Page 1 -> 7 0 1 HIT
Page 1 -> 7 0 1 HIT
Page 0 -> 7 0 1 HIT

Total Page Faults = 11
Total Page Hits = 10

LRU:

Page 5 -> 5 - - FAULT
Page 0 -> 5 0 - FAULT
Page 1 -> 5 0 1 FAULT
Page 0 -> 5 0 1 HIT
Page 2 -> 2 0 1 FAULT
Page 3 -> 2 0 3 FAULT
Page 0 -> 2 0 3 HIT
Page 2 -> 2 0 3 HIT
Page 4 -> 2 0 4 FAULT
Page 3 -> 2 3 4 FAULT
Page 3 -> 2 3 4 HIT
Page 2 -> 2 3 4 HIT
Page 0 -> 2 3 0 FAULT
Page 2 -> 2 3 0 HIT
Page 1 -> 2 1 0 FAULT
Page 2 -> 2 1 0 HIT
Page 7 -> 2 1 7 FAULT
Page 0 -> 2 0 7 FAULT
Page 1 -> 1 0 7 FAULT
Page 1 -> 1 0 7 HIT
Page 0 -> 1 0 7 HIT

Total Page Faults = 12
Total Page Hits = 9

Optimal:

Page 5 -> 5 - - FAULT
Page 0 -> 5 0 - FAULT
Page 1 -> 5 0 1 FAULT
Page 0 -> 5 0 1 HIT
Page 2 -> 2 0 1 FAULT
Page 3 -> 2 0 3 FAULT
Page 0 -> 2 0 3 HIT
Page 2 -> 2 0 3 HIT
Page 4 -> 2 4 3 FAULT
Page 3 -> 2 4 3 HIT
Page 3 -> 2 4 3 HIT
Page 2 -> 2 4 3 HIT
Page 0 -> 2 0 3 FAULT
Page 2 -> 2 0 3 HIT
Page 1 -> 2 0 1 FAULT
Page 2 -> 2 0 1 HIT
Page 7 -> 7 0 1 FAULT
Page 0 -> 7 0 1 HIT
Page 1 -> 7 0 1 HIT
Page 1 -> 7 0 1 HIT
Page 0 -> 7 0 1 HIT

Total Page Faults = 9
Total Page Hits = 12